

the output of the following one-liner?

```
perl -ne '$. == 2 && print $_' file1.txt
```

Here, only line No. 2 of the text file *file1.txt* is printed on the screen. Then what about:

```
perl -ne '$. != 2 && print $_' file1.txt
```

In this case, every line of the text file *file1.txt* except line No. 2 is printed on the screen. The trick lies with the switch `-n` of Perl. The switch `-n` will create the effect of a loop, so that the Perl script following this switch will be executed on every line of the text file being processed, thereby creating the effect of an AWK script. By the way, AWK is another text processing language still used (though Perl is much more popular) but mostly abandoned by professional programmers. Figure 2 shows the output of the Perl one-liners discussed here.

The knowledge of regular expressions will greatly increase the power of Perl one-liners. It is time for us to run a Perl script with a regular expression and for that, too, we will use the text file *file1.txt*. To process this text file, we have a Perl script called *regex1.pl*. This and all the other Perl scripts and text files used in this article can be downloaded from opensourceforu.com/article_source_code/August17perl.zip. The extension of a Perl script is *.pl* and hence the name *regex1.pl*. Now it is time to inspect and understand the code of *regex1.pl*.

```
$infile=$ARGV[0];
open (DATA,"<$infile") || die "File $infile Not Found";
while(<DATA>) {
    if(/UNIX/) {
        print "I found a match at: $_";
    }
}
```

The first line of code in the script reads the name of the text file to be processed from the command line. The second line of code opens the file specified by the command line argument if it exists, and closes the program if the file does not exist. The third line reads the whole text file, line by line, and the fourth line has an *if* statement, where a regular expression is tested to true or false. If the regular expression finds a match, the fifth line of code prints the message: 'I found a match at:' followed by the contents of the matched line on the screen.

Now, the only thing that matters is the way a regular expression is represented in a Perl script. If you observe carefully, you will see that the regular expression is placed inside a pair of forward slashes—two forward slashes acting as a delimiter for the regular expression. Since we are familiar with the Python regular expressions mentioned in the previous article, it should be clear that the regular expression match operator `/UNIX/` will result in a literal

```
# perl -e '$now=localtime(); print "$now\n"'
Fri Jun 30 23:58:26 2017
# perl -ne 'print "$. "' file1.txt
1 unix is an operating system
2 Unix is an Operating System
3 UNIX IS AN OPERATING SYSTEM
4 Linux is also an Operating System
# perl -ne '$. == 2 && print $_' file1.txt
Unix is an Operating System
# perl -ne '$. != 2 && print $_' file1.txt
unix is an operating system
UNIX IS AN OPERATING SYSTEM
Linux is also an Operating System
```

Figure 2: Output of Perl one-liners

match for the string UNIX only. Please note that if *abcd* is a regular expression, then */abcd/* is the corresponding regular expression match operator. Also, remember that the convention followed in the articles in this series is that all the regular expressions and commands are italicised to highlight them from normal text.

The line of code `print "I found a match at: $_"` replaced with the line of code `print "$."` will result in printing just the lines matched by a regular expression. This code can be found in the Perl script *regex2.pl*. This is just one way of specifying a regular expression in Perl; the regular expressions are placed inside forward slashes, like */abcd/*. Some operators like *m/*, *s/*, etc., are also used for processing text files with Perl regular expressions, and they will be discussed later in this article.

For the given text file, only one line results in a match for the regular expression `/UNIX/`. Hence a single statement, "I found a match at: UNIX IS AN OPERATING SYSTEM", is printed on screen. So, it is clear that a case-sensitive match is being carried out here in this case. How does one carry out a case-insensitive match in Perl? The line of code `if(/UNIX/)` replaced with the line of code `if(/UNIX/i)` will result in a case-insensitive match as well as a match for the words 'unix', 'Unix' and 'UNIX'. This code can be found in the Perl script *regex3.pl*. In this code, the pattern object `/UNIX/` is modified with the pattern modifier *i* for a case-insensitive match. There are other pattern modifiers also used in Perl regular expressions. The pattern modifier *s* is used to change the behaviour of the metacharacter dot (`.`). Usually, the newline character `\n` will not be matched by the dot operator but if the pattern modifier *s* is used, then even the newline character will be matched. The pattern modifier *m* will modify how metacharacters `^` and `$` are treated in case of a string with an embedded newline character.

Now, the next logical question is how to print the matched string alone and not the entire line of text, where a regular expression match is obtained. For this, we need to make a few modifications in the Perl script *regex3.pl*. This modified script, called *regex4.pl*, is shown below.

```
$infile=$ARGV[0];
open (DATA,"<$infile") || die "File $infile Not Found";
while(<DATA>) {
```